

Face Expression Recognition Using Convolutional Neural Network

Complete Project Documentation

Implementation in Google Colab using TensorFlow/Keras and the jonathanoheix/face-expression-recognition-dataset Kaggle dataset.

Project Type	Deep Learning / Computer Vision
Model	Convolutional Neural Network (CNN)
Framework	TensorFlow / Keras
Input	48 × 48 grayscale face images
Classes	7 facial-expression categories
Training	80% training / 20% validation
Epochs	40

1. Abstract

This project implements a facial-expression recognition system using a Convolutional Neural Network (CNN). The notebook downloads the Kaggle face-expression recognition dataset with KaggleHub, discovers the expression-class folders, converts images to 48×48 grayscale tensors, and creates training and validation datasets with an 80/20 split. The CNN uses data augmentation, convolutional layers, batch normalization, max pooling, dropout, and dense layers. The network is trained for 40 epochs using the Adam optimizer and sparse categorical cross-entropy loss. Evaluation includes validation accuracy/loss, confusion matrix, classification report, and actual-versus-predicted image visualization. The trained model is also saved in Keras format and can be used to predict a newly uploaded image.

2. Project Objectives

- Build a CNN-based facial-expression classification system.
- Load and inspect a facial-expression image dataset in Google Colab.
- Prepare images as 48×48 grayscale inputs.
- Use data augmentation to improve model generalization.
- Train a multi-class CNN for seven expression categories.
- Evaluate the model using accuracy, loss, confusion matrix, and classification report.
- Compare actual and predicted expressions visually.
- Save the trained model for later inference.

3. Tools and Technologies

Technology	Purpose
Python	Programming language used in the notebook
Google Colab	Cloud notebook/training environment
KaggleHub	Dataset download
TensorFlow / Keras	CNN construction and training
NumPy	Numerical operations and labels
Pandas	Tabular result handling
Matplotlib	Plots and image visualization
Seaborn	Confusion-matrix visualization
scikit-learn	Confusion matrix and classification report
Pillow (PIL)	Image inspection

4. Dataset

The notebook downloads the dataset using the KaggleHub identifier **jonathanoheix/face-expression-recognition-dataset**. The code searches for the directory containing the seven expected class folders and then counts images in each class.

4.1 Expression Classes

- Angry
- Disgust

- Fear
- Happy
- Neutral
- Sad
- Surprise

The exact number of images in each class is calculated at runtime by the notebook and displayed as a class-distribution bar chart. This documentation does not invent those counts because the uploaded notebook contains the counting code but not its executed output.

4.2 Dataset Preparation

- Image size: 48 × 48 pixels.
- Color mode: grayscale (one channel).
- Batch size: 64.
- Random seed: 42.
- Training split: 80%.
- Validation split: 20%.
- Labels: integer class IDs inferred from directory names.
- Training data is shuffled; validation data is not shuffled.

5. System Workflow

Dataset Download → Dataset Inspection → Class Detection → Image Preprocessing → Train/Validation Split → Data Augmentation → CNN → Training → Validation → Confusion Matrix / Classification Report → Actual vs Predicted → New Image Prediction → Model Saving

6. Data Preprocessing

TensorFlow's `image_dataset_from_directory` is used to load the directory-organized dataset. The images are resized to 48 × 48 and converted to grayscale. Integer labels are inferred from the class directories. The input tensors are later normalized by a Keras Rescaling layer.

```
IMG_SIZE = (48, 48)
BATCH_SIZE = 64
SEED = 42

train_ds = tf.keras.utils.image_dataset_from_directory(
    dataset_dir,
    labels='inferred',
    label_mode='int',
    color_mode='grayscale',
    batch_size=BATCH_SIZE,
    image_size=IMG_SIZE,
    validation_split=0.2,
    subset='training',
    seed=SEED,
    shuffle=True
)
```

7. Data Augmentation

The CNN includes a Keras augmentation pipeline. During training, images can be randomly flipped horizontally, rotated, zoomed, and translated. These transformations create variation in the training input and can help reduce

overfitting.

```
data_augmentation = keras.Sequential([
    layers.RandomFlip("horizontal"),
    layers.RandomRotation(0.08),
    layers.RandomZoom(0.10),
    layers.RandomTranslation(
        height_factor=0.05,
        width_factor=0.05
    )
])
```

8. CNN Architecture

The model is a sequential CNN. It accepts a $48 \times 48 \times 1$ grayscale image. The network first applies augmentation and rescales pixels by $1/255$. It then uses three convolutional blocks with increasing filter counts: 32, 64, and 128. Each block contains convolutional layers, batch normalization, max pooling, and dropout. The extracted feature maps are flattened and passed to a 128-unit dense layer followed by a seven-unit softmax output.

Stage	Main components
Input	$48 \times 48 \times 1$
Preprocessing	Data augmentation + Rescaling($1/255$)
Block 1	Conv2D 32 → BatchNorm → Conv2D 32 → MaxPool → Dropout 0.25
Block 2	Conv2D 64 → BatchNorm → Conv2D 64 → MaxPool → Dropout 0.25
Block 3	Conv2D 128 → BatchNorm → Conv2D 128 → MaxPool → Dropout 0.30
Classifier	Flatten → Dense 128 → BatchNorm → Dropout 0.50
Output	Dense 7 → Softmax

9. Model Compilation and Training

The model is compiled with the Adam optimizer using a learning rate of 0.001. Because the labels are integer class IDs, sparse categorical cross-entropy is used as the loss function. Accuracy is tracked as the primary metric.

```
model.compile(
    optimizer=keras.optimizers.Adam(
        learning_rate=0.001
    ),
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)
```

Training is configured for 40 epochs. The notebook uses ReduceLROnPlateau to reduce the learning rate when validation loss stops improving, and ModelCheckpoint to save the model with the best validation accuracy. EarlyStopping is not used in the uploaded notebook, so the configured training run proceeds through the full 40 epochs unless an external runtime failure interrupts it.

10. Training Configuration

Parameter	Value
Epochs	40
Optimizer	Adam
Initial learning rate	0.001
Loss	Sparse categorical cross-entropy
Metric	Accuracy
Learning-rate scheduler	ReduceLROnPlateau

Parameter	Value
LR reduction factor	0.5
LR patience	3 epochs
Minimum learning rate	1e-6
Checkpoint monitor	Validation accuracy
Checkpoint file	best_face_expression_model.keras

11. Model Evaluation

After training, the notebook evaluates the model on the validation dataset. It records validation loss and validation accuracy, plots training-versus-validation accuracy and loss, and generates predictions for every validation batch.

11.1 Accuracy and Loss

The notebook plots training accuracy against validation accuracy and training loss against validation loss. These plots are used to observe learning behavior and possible overfitting. The numerical results depend on the actual Colab training run and are therefore not fabricated in this document.

11.2 Confusion Matrix

A seven-by-seven confusion matrix is generated. Rows represent actual expressions and columns represent predicted expressions. Diagonal entries represent correct classifications, while off-diagonal entries show where expressions were confused.

11.3 Classification Report

The scikit-learn `classification_report` provides precision, recall, and F1-score for the available classes. The notebook also uses `zero_division=0` to avoid undefined-metric warnings when a class has no predicted samples.

12. Actual vs Predicted Analysis

The notebook creates predictions for the validation images and displays image examples with their actual and predicted expression labels. This provides a qualitative check of model behavior in addition to numerical metrics. The prediction arrays are stored as `y_true` and `y_pred`, where each value is a class index.

```
y_true = []
y_pred = []

for images, labels in val_ds:
    predictions = model.predict(images, verbose=0)
    predicted_labels = np.argmax(predictions, axis=1)

    y_true.extend(labels.numpy())
    y_pred.extend(predicted_labels)

y_true = np.array(y_true)
y_pred = np.array(y_pred)
```

13. Single-Image Prediction

The project also supports inference on an image uploaded through Google Colab. The image is loaded as grayscale, resized to 48×48 , converted to an array, given a batch dimension, and passed to the CNN. The predicted class is the output neuron with the highest softmax probability. The notebook also reports the corresponding confidence value.

14. Model Saving

The final model is saved in the native Keras format. This allows the trained network to be loaded later without rebuilding the architecture.

```
model.save(  
    "face_expression_cnn.keras"  
)
```

A separate checkpoint named `best_face_expression_model.keras` is also created during training when validation accuracy improves.

15. Expected Output and Deliverables

- Dataset directory and class-distribution information.
- Sample training images.
- Training and validation accuracy graph.
- Training and validation loss graph.
- Validation loss and accuracy.
- Confusion matrix.
- Classification report.
- Actual-versus-predicted sample images.
- Single-image expression prediction with confidence.
- Saved CNN model: `face_expression_cnn.keras`.
- Best-validation-accuracy checkpoint: `best_face_expression_model.keras`.

16. Limitations

- The notebook uses a validation split rather than a separately loaded independent test set.
- The uploaded notebook does not contain executed metric outputs, so this documentation does not claim a specific final accuracy, precision, recall, or F1-score.
- The system classifies images into the seven trained expression categories; it does not identify a person's identity.
- Predicted expression labels are model classifications and should not be treated as definitive measurements of a person's internal emotional state.
- Performance can vary with lighting, pose, image quality, cropping, demographic variation, and differences between the training images and new images.

17. Possible Future Improvements

- Create a separate held-out test set in addition to the validation set.
- Use class weights or other balancing strategies if the observed class distribution is highly uneven.
- Experiment with different CNN depths, filter sizes, dropout values, and learning rates.
- Compare the custom CNN with transfer-learning approaches.
- Add face detection/cropping before expression classification for unconstrained images.
- Evaluate the model on images from a different source to measure generalization.
- Save and compare per-class metrics across multiple training runs.

18. Conclusion

The project provides a complete end-to-end CNN pipeline for facial-expression recognition. It covers dataset acquisition, inspection, preprocessing, augmentation, CNN design, training, evaluation, visualization, inference, and model persistence. The implementation is suitable as a practical deep-learning project demonstrating multi-class image classification with TensorFlow/Keras. Final performance should be reported from the actual executed Colab run rather than estimated from the source code alone.

19. Reproducibility Notes

To reproduce the project, run the notebook from the first cell in Google Colab with the required packages available. The dataset is downloaded at runtime through KaggleHub. The random seed is set to 42 for the directory split, but exact training results can still vary with TensorFlow version, hardware, runtime behavior, and other sources of nondeterminism.

20. Appendix — Core CNN Code

```
model = Sequential([
    layers.Input(shape=(48, 48, 1)),
    data_augmentation,
    layers.Rescaling(1./255),

    layers.Conv2D(32, (3, 3), padding='same', activation='relu'),
    layers.BatchNormalization(),
    layers.Conv2D(32, (3, 3), padding='same', activation='relu'),
    layers.MaxPooling2D((2, 2)),
    layers.Dropout(0.25),

    layers.Conv2D(64, (3, 3), padding='same', activation='relu'),
    layers.BatchNormalization(),
    layers.Conv2D(64, (3, 3), padding='same', activation='relu'),
    layers.MaxPooling2D((2, 2)),
    layers.Dropout(0.25),

    layers.Conv2D(128, (3, 3), padding='same', activation='relu'),
    layers.BatchNormalization(),
    layers.Conv2D(128, (3, 3), padding='same', activation='relu'),
    layers.MaxPooling2D((2, 2)),
    layers.Dropout(0.30),

    layers.Flatten(),
    layers.Dense(128, activation='relu'),
    layers.BatchNormalization(),
    layers.Dropout(0.50),

    layers.Dense(7, activation='softmax')
])
```